

All your favorite parts of Medium are now in one sidebar for easy access.

access to the best of Medium for less than \$1/week. [Become a member](#)



Okay, got it

Production-Ready Observability with Prometheus, Loki & Grafana



Neamul Kabir Emon

Follow

11 min read · Apr 18, 2025



3



In this project, you'll learn how to build a **complete observability stack on Kubernetes** using industry-standard tools like **Prometheus, Loki, and**

Grafana, integrated with a lightweight but powerful demo app: **Podinfo**.

All your favorite parts of
Medium are now in one
sidebar for easy access.

real-world production environments and gives you hands-

 Real-time metrics and performance dashboards

-  Centralized structured logging
-  Cross-layer insights with metrics and logs combined

Whether you're an SRE, DevOps engineer, or platform developer — this project will help you understand **how to monitor, debug, and operate modern cloud-native workloads** effectively.

GitHub Repo → <https://github.com/neamulkabiremon/k8s-monitoring-project.git>

Tools Used

- **Helm** — For deploying Kubernetes apps
- **Prometheus** — For collecting metrics
- **Loki + Promtail** — For centralised logging
- **Grafana** — For dashboards and visualisation
- **Podinfo** — Sample app with built-in observability

Step 1: Getting Started

All your favorite parts of Medium are now in one sidebar for easy access.

On a Kubernetes cluster (GKE), deploy the observability stack, a sample app, and finally **create production-grade** dashboards for both metrics and logs. This setup is perfect for learning, demos, or as a foundation for real-world monitoring solutions. Let's get started!

1 Clone the Repository

```
git clone https://github.com/neamulkabiremon/k8s-monitoring-project.git
cd k8s-monitoring-project
```

This repository contains all the infrastructure code, Kubernetes manifests, Argo CD apps, and observability tools needed for a production-grade setup.

2 Provision the EKS Cluster with Terraform

```
cd terraform/gke-cluster
```

Initialise the Terraform working directory:

```
terraform init
```

```
~/d/monitoring/k8s-monitoring-project/terraform/gke-cluster main !22 ?1
terraform init

Initializing the backend...

All your favorite parts of Medium are now in one sidebar for easy access.

...
of hashicorp/google from the dependency lock file
of hashicorp/helm from the dependency lock file
le v6.28.0...
e v6.28.0 (signed by HashiCorp)
v2.17.0...
v2.17.0 (signed by HashiCorp)

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Apply the configuration to provision the Kubernetes cluster:

```
terraform apply -auto-approve
```

```
+ verify = false
+ version = "4.12.1"
+ wait = true
+ wait_for_jobs = false

+ set {
  + name = "controller.publishService.enabled"
  + value = "true"
}

Plan: 14 to add, 0 to change, 0 to destroy.
google_service_account.gke: Creating...
google_compute_firewall.allow_iap_ssh: Creating...
google_compute_route.default_route: Creating...
google_compute_subnetwork.public: Creating...
google_compute_router.router: Creating...
google_compute_address.nat: Creating...
google_compute_subnetwork.private: Creating...
google_compute_address.nat: Creation complete after 8s [id=projects/serious-physics-452107-d1/regions/us-cent
addresses/nat]
google_compute_firewall.allow_iap_ssh: Still creating... [10s elapsed]
google_compute_route.default_route: Still creating... [10s elapsed]
google_service_account.gke: Still creating... [10s elapsed]
google_compute_subnetwork.public: Still creating... [10s elapsed]
google_compute_router.router: Still creating... [10s elapsed]
google_compute_subnetwork.private: Still creating... [10s elapsed]
google_compute_route.default_route: Creation complete after 12s [id=projects/serious-physics-452107-d1/global
```

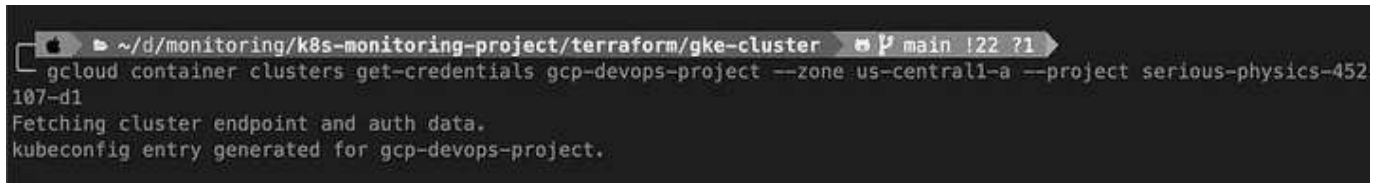
 **Note:** This process takes approximately 10–20 minutes, depending on your cloud provider, region and network conditions.

All your favorite parts of
Medium are now in one
sidebar for easy access.

th the EKS Cluster

Once `terraform` completes successfully, authenticate your local `kubectl` context with the newly created GKE Kubernetes cluster:

```
ials gcp-devops-project --zone us-central1-a --project serious-physics-452107-d1
```



```
~/d/monitoring/k8s-monitoring-project/terraform/gke-cluster main !22 ?1  
gcloud container clusters get-credentials gcp-devops-project --zone us-central1-a --project serious-physics-452107-d1  
Fetching cluster endpoint and auth data.  
kubeconfig entry generated for gcp-devops-project.
```

☒ Replace `<your-region>` (e.g., `us-east-1`) and `<your-cluster-name>` (e.g., `ultimate-devops-cluster`) with your actual configuration values.

4 Verify Cluster Access

Check if your Kubernetes context is properly configured and the cluster is accessible:

```
kubectl get nodes
```

```
~/d/monitoring/k8s-monitoring-project/terraform/gke-cluster main !22 ?1
kubectll get nodes
NAME                                STATUS    ROLES    AGE     VERSION
gke-node-pool-603d92ee-cgbv        Ready    <none>    5m40s   v1.31.6-gke.1020000
```

All your favorite parts of
Medium are now in one
sidebar for easy access.

of nodes with a STATUS of Ready. This confirms that
ected and the cluster is up and running.

Step 2: Install the Observability Stack

✓ Step 1: One-Click Observability Installation

To make the installation process seamless, I've created a bash script that
automates everything:

📁 [grafana-prom-loki.sh](#)

```
#!/bin/bash

# Create namespace
kubectl create namespace monitoring || true

# Add helm repos
helm repo add prometheus-community https://prometheus-community.github.io/helm-c
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

# Install Loki stack (loki + promtail)
helm install loki grafana/loki-stack \
  --namespace monitoring \
  --set loki.enabled=true,promtail.enabled=true

# Install kube-prometheus-stack with Grafana, Prometheus, Alertmanager
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  -f ./custom_kube_prometheus_stack.yml
```

Step 2: Custom Configuration with

All your favorite parts of Medium are now in one sidebar for easy access.

ires that **Grafana, Prometheus, and Alertmanager** are set up with high availability and the required data sources.

YAML:

custom_kube_prometheus_stack.yml

```
alertmanager:
  alertmanagerSpec:
    alertmanagerConfigSelector:
      matchLabels:
        release: monitoring
    replicas: 2
    alertmanagerConfigMatcherStrategy:
      type: None
```

- ☒ **replicas: 2** — ensures high availability of alerting service
- ☒ **matchLabels** — links Alertmanager with its CRDs
- ☒ **None strategy** — avoids complex matching pitfalls

Grafana: Auto-Add Prometheus & Loki as Data Sources

```
grafana:
  additionalDataSources:
    - name: Loki
      type: loki
      url: http://loki:3100
      access: proxy
      jsonData:
        maxLines: 1000
```

-  **Loki** — Enables real-time log exploration in Grafana

-  **Prometheus** — Scrapes and visualizes application metrics

All your favorite parts of
Medium are now in one
sidebar for easy access.

ie — Makes Prometheus the default data source for

☒ Run this script with:

```
chmod +x grafana-prom-loki.sh
./grafana-prom-loki.sh
```

```
~/desktop/monitoring/k8s-monitoring-project 08:28:49 PM
chmod +x grafana-prom-loki.sh

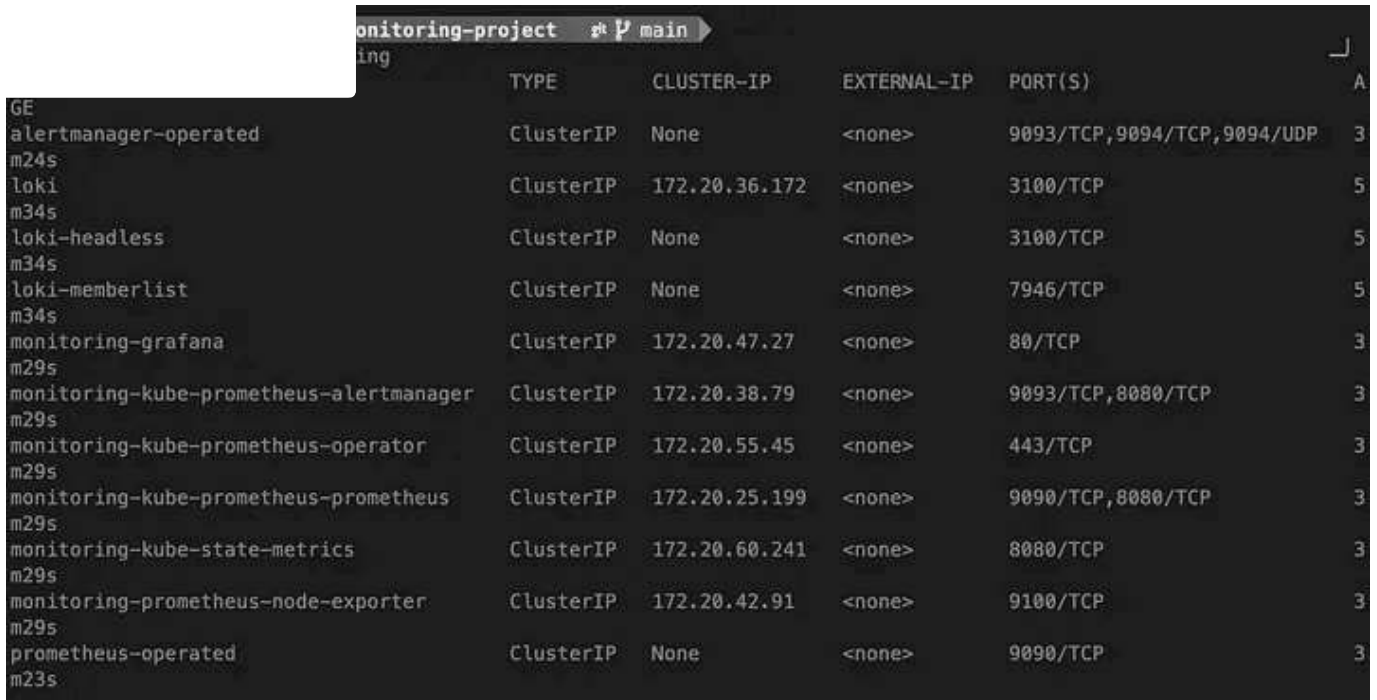
~/desktop/monitoring/k8s-monitoring-project 08:29:18 PM
./grafana-prom-loki.sh
namespace/monitoring created
"prometheus-community" already exists with the same configuration, skipping
"grafana" already exists with the same configuration, skipping
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "jetstack" chart repository
...Successfully got an update from the "ingress-nginx" chart repository
...Successfully got an update from the "podinfo" chart repository
...Successfully got an update from the "metrics-server" chart repository
...Successfully got an update from the "autoscaler" chart repository
...Successfully got an update from the "secrets-store-csi-driver" chart repository
...Successfully got an update from the "aws-load-balancer-controller" chart repository
...Successfully got an update from the "eks" chart repository
...Successfully got an update from the "open-telemetry" chart repository
...Successfully got an update from the "gitlab" chart repository
...Successfully got an update from the "bitnami" chart repository
...Successfully got an update from the "grafana" chart repository
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. *Happy Helming!*
NAME: loki
LAST DEPLOYED: Wed Apr 9 20:29:44 2025
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
NOTES:
The Loki stack has been deployed to your cluster. Loki can now be added as a datasource in Grafana.
```

☒ Step 3: Verify Installation

Once the script completes, run:

```
kubectl get svc -n monitoring
```

All your favorite parts of Medium are now in one sidebar for easy access.



The screenshot shows a terminal window with the command `kubectl get svc -n monitoring` executed. The output is a table of services in the monitoring namespace. The table has columns for NAME, TYPE, CLUSTER-IP, EXTERNAL-IP, PORT(S), and AGE. The services listed are alertmanager-operated, loki, loki-headless, loki-memberlist, monitoring-grafana, monitoring-kube-prometheus-alertmanager, monitoring-kube-prometheus-operator, monitoring-kube-prometheus-prometheus, monitoring-kube-state-metrics, monitoring-prometheus-node-exporter, and prometheus-operated.

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
alertmanager-operated	ClusterIP	None	<none>	9093/TCP,9094/TCP,9094/UDP	3m24s
loki	ClusterIP	172.20.36.172	<none>	3100/TCP	5m34s
loki-headless	ClusterIP	None	<none>	3100/TCP	5m34s
loki-memberlist	ClusterIP	None	<none>	7946/TCP	5m34s
monitoring-grafana	ClusterIP	172.20.47.27	<none>	80/TCP	3m29s
monitoring-kube-prometheus-alertmanager	ClusterIP	172.20.38.79	<none>	9093/TCP,8080/TCP	3m29s
monitoring-kube-prometheus-operator	ClusterIP	172.20.55.45	<none>	443/TCP	3m29s
monitoring-kube-prometheus-prometheus	ClusterIP	172.20.25.199	<none>	9090/TCP,8080/TCP	3m29s
monitoring-kube-state-metrics	ClusterIP	172.20.60.241	<none>	8080/TCP	3m29s
monitoring-prometheus-node-exporter	ClusterIP	172.20.42.91	<none>	9100/TCP	3m29s
prometheus-operated	ClusterIP	None	<none>	9090/TCP	3m23s

You should see services like:

- monitoring-grafana
- monitoring-kube-prometheus-prometheus
- loki

Expose Grafana

Patch the Grafana service to make it accessible externally:

```
kubectl patch svc monitoring-grafana -n monitoring -p '{"spec": {"type": "LoadBalancer", "ports": [{"name": "http", "port": 80, "targetPort": 3000}], "selector": {"app": "monitoring-grafana"}, "type": "LoadBalancer"}' --dry-run
```

Change 'type: ClusterIP' to 'LoadBalancer'

Check the new external IP:

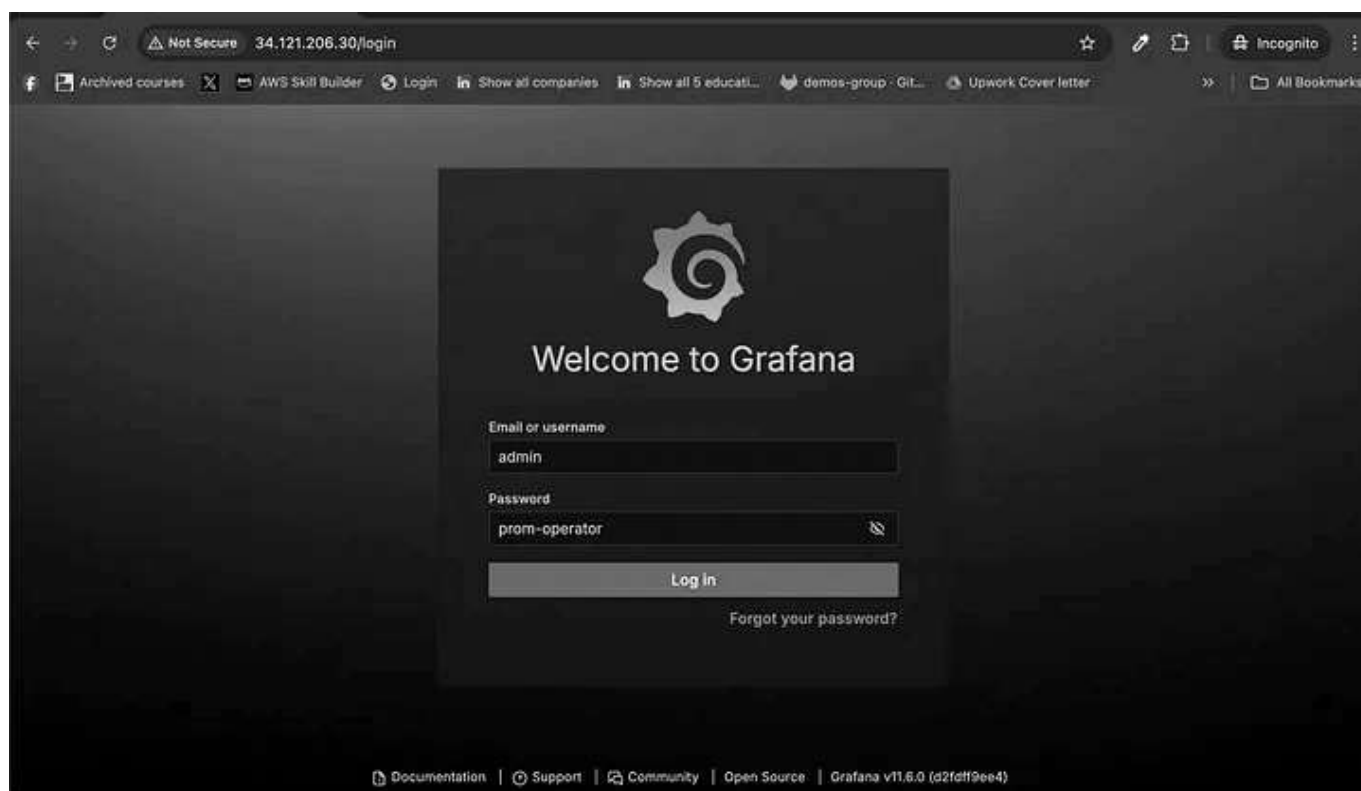
All your favorite parts of Medium are now in one sidebar for easy access.

```
monitoring-grafana -n monitoring
```

You should see the External LoadBalancer IP Address:

```
~/desktop/monitoring/k8s-monitoring-project  main  gcp-devops-project  12:44:07 PM
kubectll get svc monitoring-grafana -n monitoring
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
monitoring-grafana  LoadBalancer  172.20.47.27  34.121.206.30  80:32243/TCP     6m6s
```

Copy the LoadBalancer EXTERNAL-IP and log in to Grafana:



Access Grafana via browser using the external IP.

- **Username:** admin

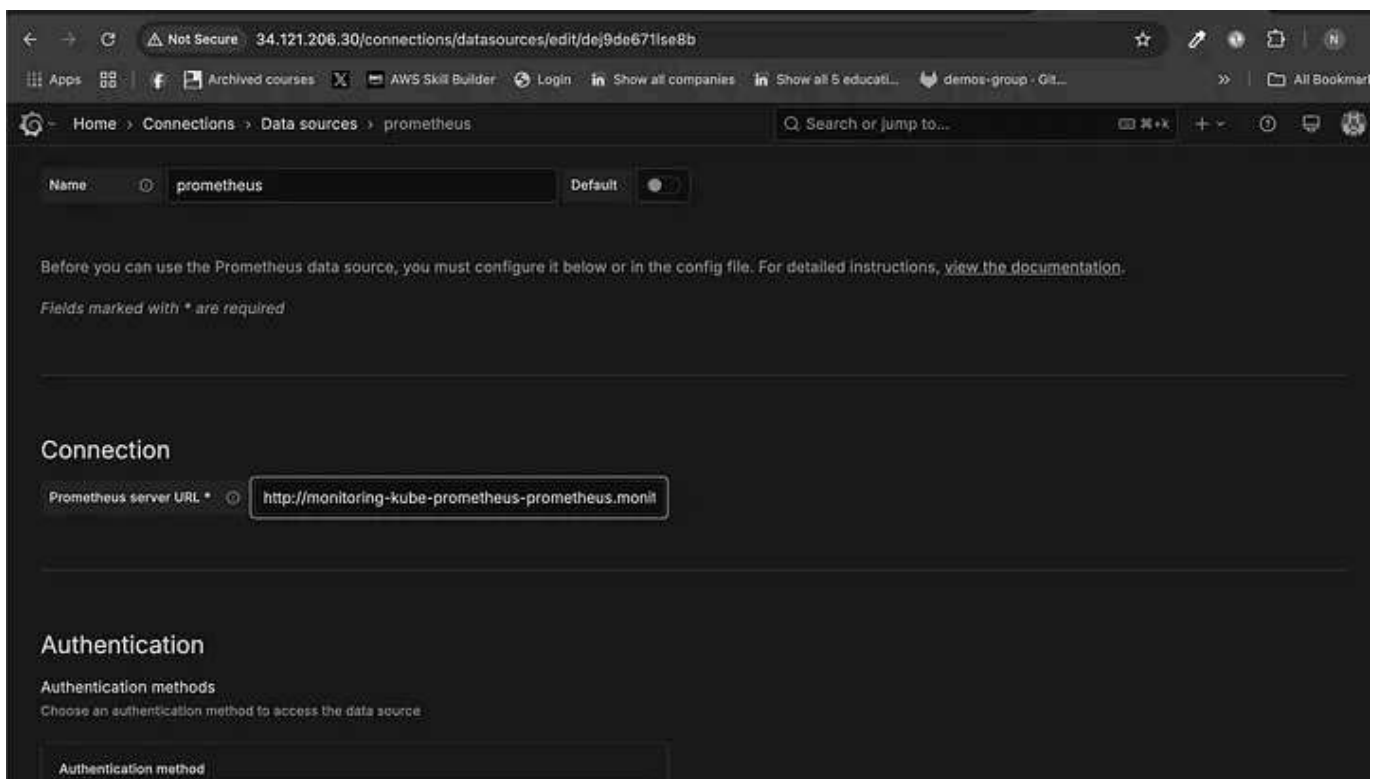
Recommended: Prometheus operator

All your favorite parts of
Medium are now in one
sidebar for easy access.

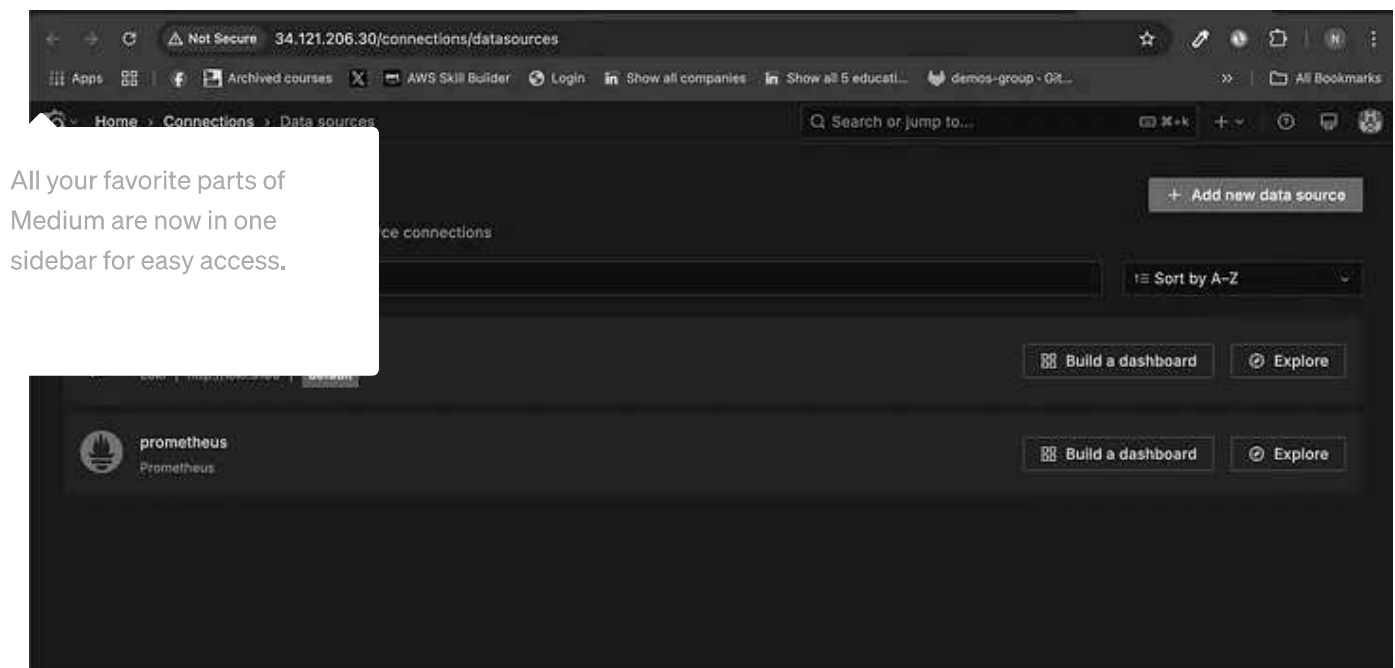
Prometheus Data Source in Grafana

- Navigate to **Settings → Data Sources → Prometheus**
- Add this internal Prometheus endpoint:

```
http://monitoring-kube-prometheus-prometheus.monitoring.svc.cluster.local:9090
```



Save and test the connection. ☒ You're now collecting metrics!



 **Note:** Loki (for logs) is already auto-configured via the install script.

Step 3: Deploy the Demo Application (Podinfo)

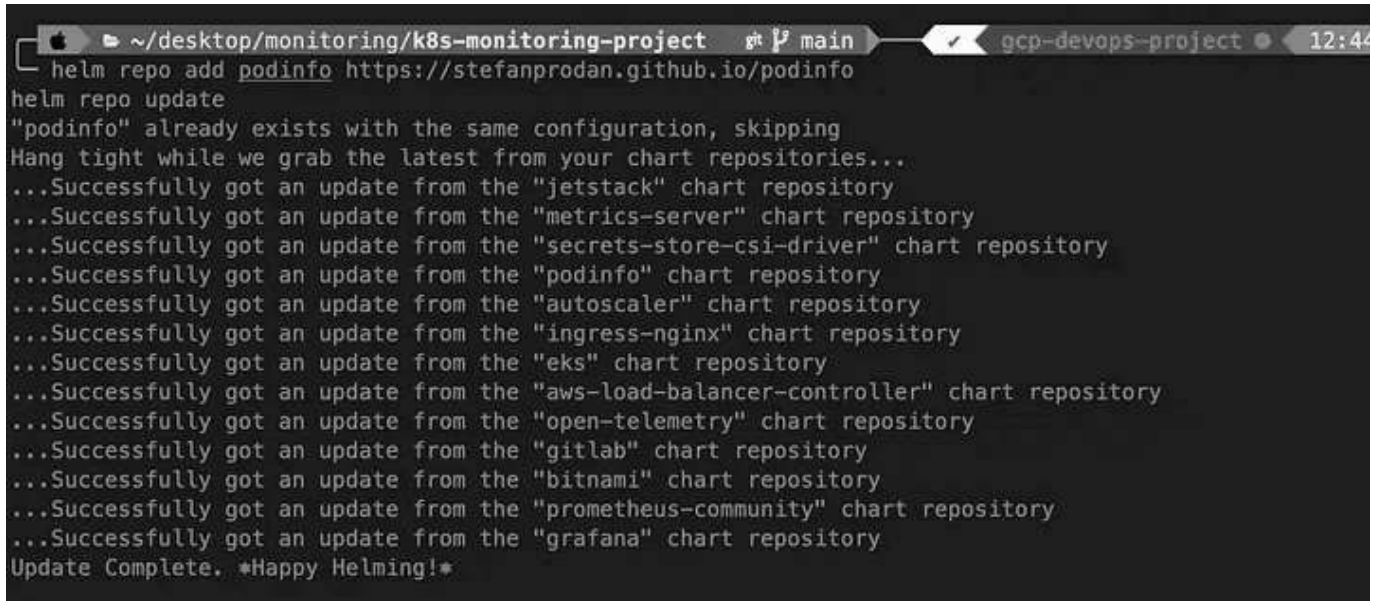
In this section, we'll deploy the Podinfo application in our Kubernetes cluster as a demo microservice. It's lightweight, production-ready, and comes pre-instrumented with **Prometheus**, **OpenTelemetry**, and structured logging — perfect for observability testing with Grafana, Loki, and Prometheus.

Step 1: Add the Helm Repository

First, add the official Helm chart repository for Podinfo:

All your favorite parts of Medium are now in one sidebar for easy access.

```
info https://stefanprodan.github.io/podinfo
```

A terminal window with a dark background. The title bar shows the file path ~/desktop/monitoring/k8s-monitoring-project, a Git icon, the branch main, and a tab for gcp-devops-project. The terminal text shows the command 'helm repo add podinfo https://stefanprodan.github.io/podinfo' being executed. This is followed by 'helm repo update', which outputs that 'podinfo' already exists and is skipped. Then, 'Hang tight while we grab the latest from your chart repositories...' is shown, followed by a list of 12 repositories being updated successfully, including 'jetstack', 'metrics-server', 'secrets-store-csi-driver', 'podinfo', 'autoscaler', 'ingress-nginx', 'eks', 'aws-load-balancer-controller', 'open-telemetry', 'gitlab', 'bitnami', 'prometheus-community', and 'grafana'. The terminal ends with 'Update Complete. *Happy Helming!*'.

Step 2: Create Namespace (Optional but Recommended)

To keep resources isolated and organised, we'll deploy **Podinfo** in its own namespace:

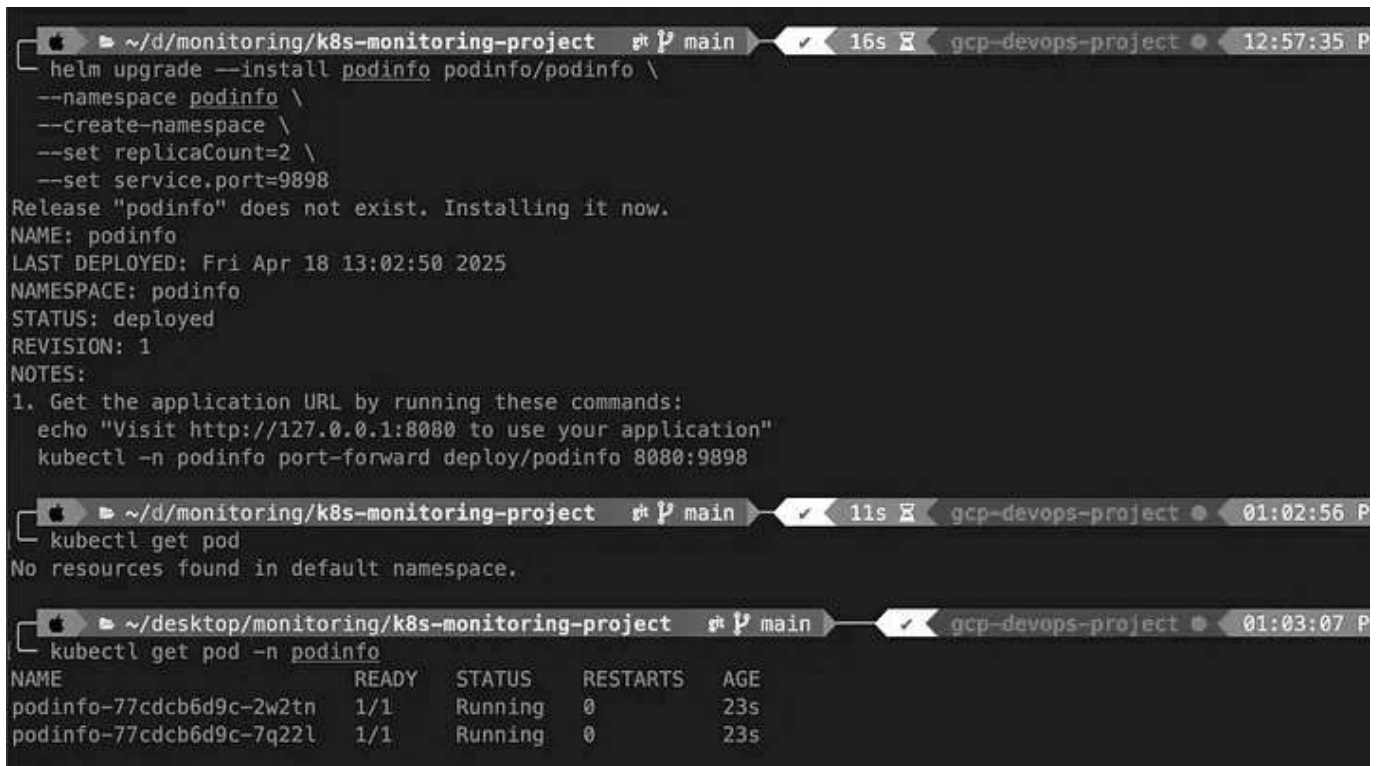
```
kubectl create namespace podinfo
```

Step 3: Deploy Podinfo with Helm

Use the official Helm chart to deploy Podinfo into the podinfo namespace:

```
helm upgrade --install podinfo podinfo/podinfo \
--namespace podinfo \
--create-namespace \
--set replicaCount=2 \
--set service.port=9898
```

All your favorite parts of
Medium are now in one
sidebar for easy access.



```
~/d/monitoring/k8s-monitoring-project main 16s gcp-devops-project 12:57:35 P
helm upgrade --install podinfo podinfo/podinfo \
--namespace podinfo \
--create-namespace \
--set replicaCount=2 \
--set service.port=9898
Release "podinfo" does not exist. Installing it now.
NAME: podinfo
LAST DEPLOYED: Fri Apr 18 13:02:50 2025
NAMESPACE: podinfo
STATUS: deployed
REVISION: 1
NOTES:
1. Get the application URL by running these commands:
  echo "Visit http://127.0.0.1:8080 to use your application"
  kubectl -n podinfo port-forward deploy/podinfo 8080:9898

~/d/monitoring/k8s-monitoring-project main 11s gcp-devops-project 01:02:56 P
kubectl get pod
No resources found in default namespace.

~/desktop/monitoring/k8s-monitoring-project main gcp-devops-project 01:03:07 P
kubectl get pod -n podinfo
NAME                                READY   STATUS    RESTARTS   AGE
podinfo-77cdcb6d9c-2w2tn            1/1     Running   0           23s
podinfo-77cdcb6d9c-7q22l            1/1     Running   0           23s
```

This setup will:

- Deploy **2 replicas** for basic high availability
- Expose the HTTP service on **port 9898**
- Rely on Prometheus **ServiceMonitor** for metrics discovery (no need for annotations)

☒ Step 4: Verify Podinfo Deployment

Check if the Podinfo pods are running successfully:

All your favorite parts of Medium are now in one sidebar for easy access.

```
-n podinfo
```

```
kubectl get pod -n podinfo
```

NAME	READY	STATUS	RESTARTS	AGE
podinfo-77cdcb6d9c-2w2tn	1/1	Running	0	23s
podinfo-77cdcb6d9c-7q22l	1/1	Running	0	23s

You should see both replicas in a Running state.

🌐 Step 5: Access the Podinfo App (via Port Forwarding)

You can access the app locally by forwarding the service port:

```
kubectl port-forward svc/podinfo 8080:9898 -n podinfo
```

```
kubectl port-forward svc/podinfo 8080:9898 -n podinfo
```

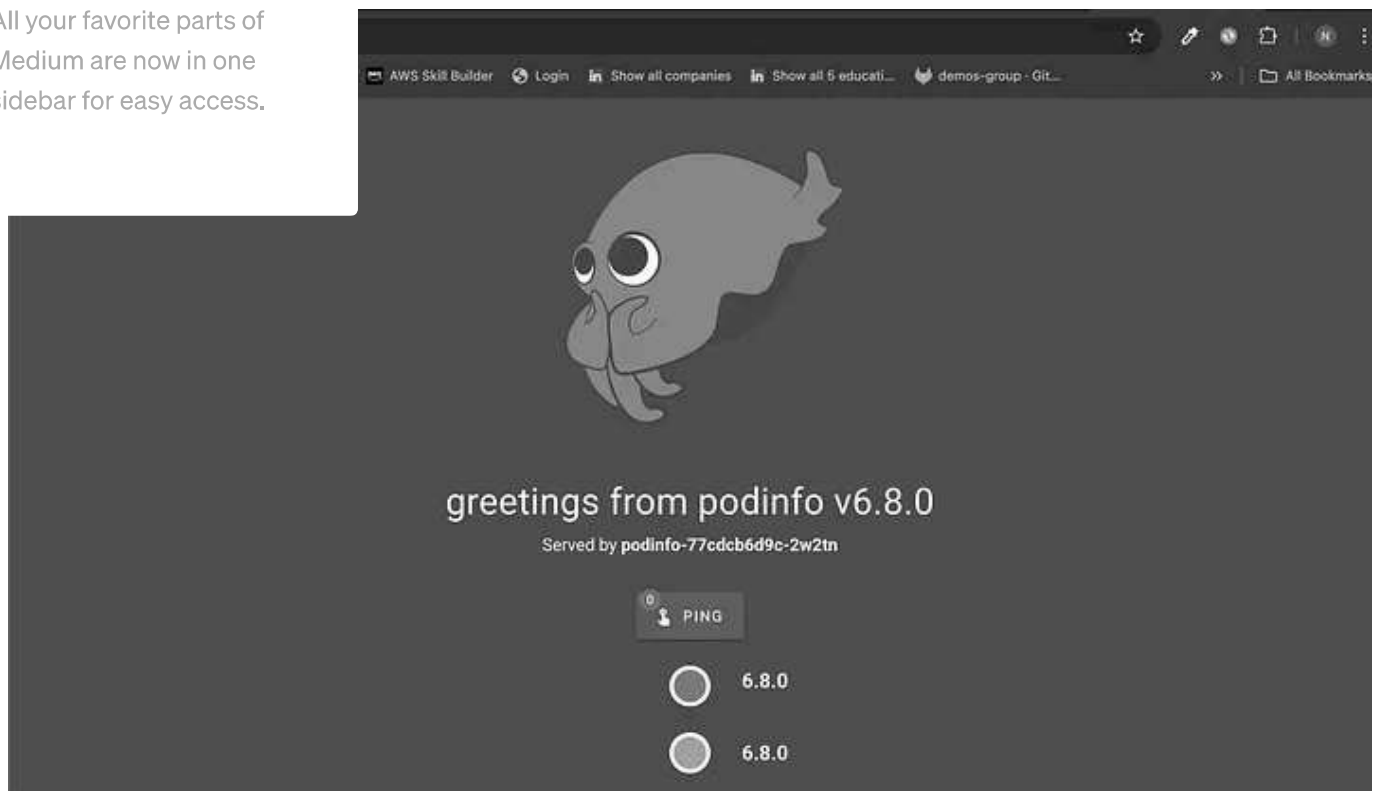
```
Forwarding from 127.0.0.1:8080 -> 9898
Forwarding from [::1]:8080 -> 9898
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

This maps: localhost:8080 → podinfo service port 9898

Now open your browser and navigate to:

<http://127.0.0.1:8080>

All your favorite parts of Medium are now in one sidebar for easy access.



You should see the **Podinfo** web UI running. ☒ The Application is running

Step 4: Configure Prometheus to Scrape Podinfo

To enable Prometheus to scrape metrics from the app, define a **ServiceMonitor** resource:

podinfo-servicemonitor.yaml

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
```

```
metadata:
  name: podinfo-servicemonitor
  namespace: monitoring
```

All your favorite parts of `monitoring` # must match your Prometheus release label
Medium are now in one
sidebar for easy access.

```
      - coreos.com/monitoring.k8s.io/name: podinfo # must match the service labels
namespaceSelector:
  matchNames:
    - podinfo # the namespace where podinfo is deployed
endpoints:
  - port: http # matches the port name in the service
    path: /metrics
    interval: 15s
```

Apply it:

```
kubectl apply -f podinfo-servicemonitor.yaml
```

```
~/d/monitoring/ultimate-devops-project-demo P main !1 ?1
kubectll apply -f observability/service-monitors/opentelemetry-demo-servicemonitor.yaml
servicemonitor.monitoring.coreos.com/opentelemetry-demo-frontendproxy-monitor created
```

✓ **Success!**

You now have a full observability pipeline set up:

- 🔍 Metrics via **Prometheus**
- 📊 Dashboards in **Grafana**
- 📄 Centralized logs via **Loki**
- 🔧 Sample app to test real-world scenarios

Step 5: Simulate Traffic and Visualize Metrics

Before we dive into Grafana, let's simulate real-world traffic so we can

All your favorite parts of Medium are now in one sidebar for easy access. wing into our dashboards.

Access to all of it.

Become a Medium member for unlimited reading.

Upgrade now

Generate Load on Podinfo

Run the provided traffic generator script to create realistic traffic patterns:

```
chmod +x generate-traffic.sh
./generate-traffic.sh
```

```
~/desktop/monitoring/k8s-monitoring-project main !2 TSTP x 38s 01:36:11 PM
chmod +x generate-traffic.sh
./generate-traffic.sh

Target Podinfo URL: http://127.0.0.1:8080
Simulating realistic traffic with metrics, logs, and traces
Press Ctrl+C to stop.

Stats for last 5 seconds:
✓ Success: 5
✗ Failures: 0
📦 Total Requests: 5

Stats for last 5 seconds:
✓ Success: 3
✗ Failures: 2
📦 Total Requests: 5
```

This script continuously sends requests to various endpoints of the Podinfo app, including:

- Health checks and environment details

Simulated 200, 404, and 500 responses

All your favorite parts of
Medium are now in one
sidebar for easy access.

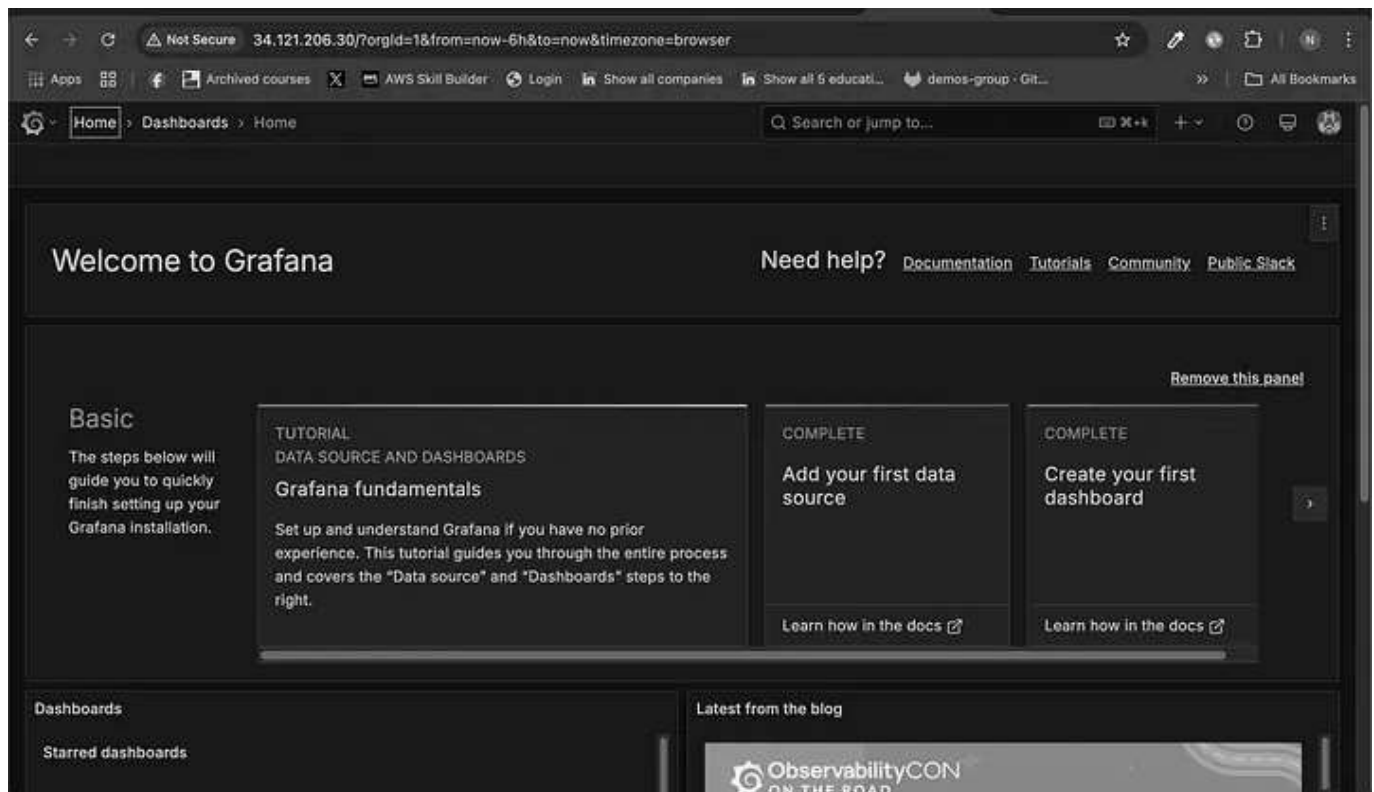
ked responses

or logging and caching

- Token issuance and validation

6: Import & Customize Grafana Dashboards

Then open Grafana <http://34.121.206.30/> in your browser.



- Username: admin
- Password: admin (or the one you set)

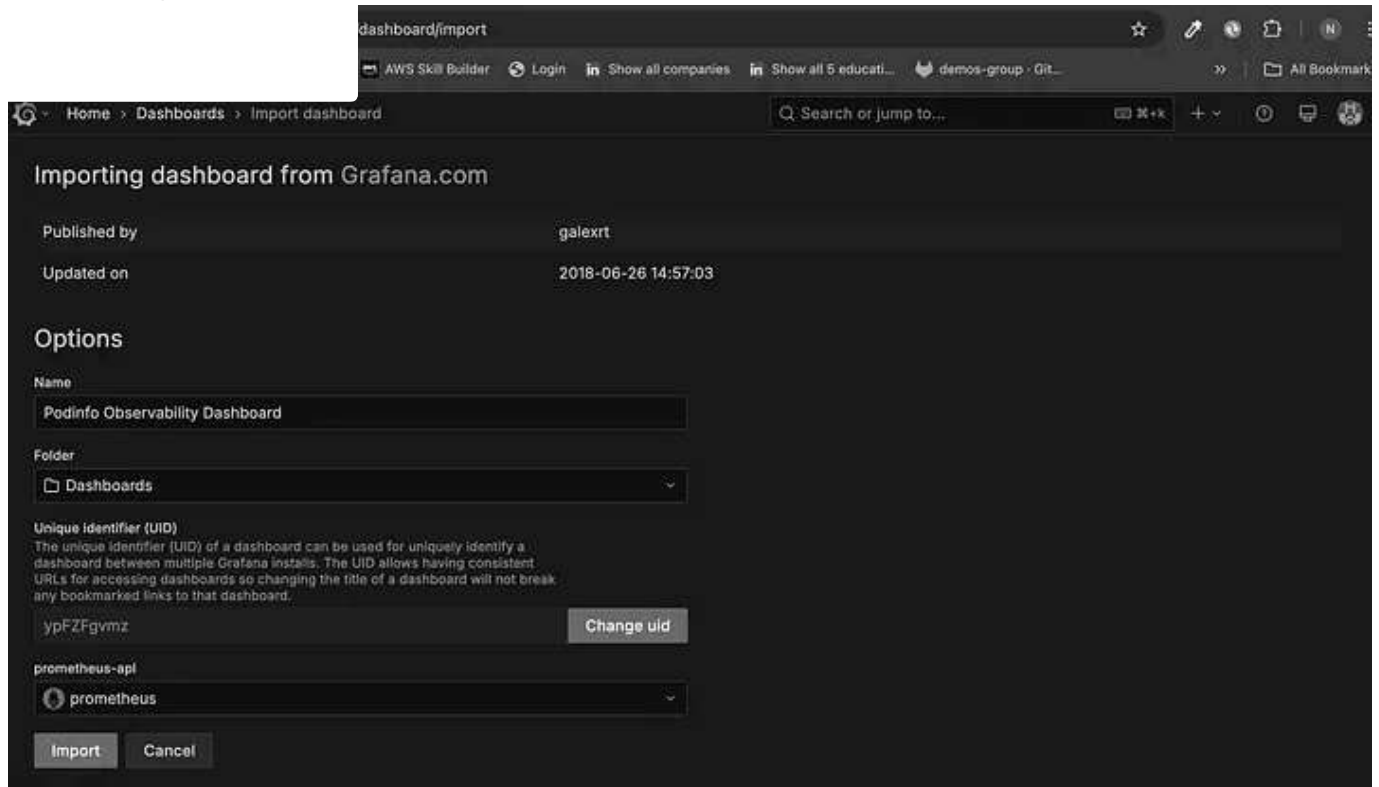
6.1. Import a Production-Ready Dashboard

1. In Grafana, click the "+" → Import.

2. Use the dashboard ID 6671 (or search for “Go Processes”).

3. Select Prometheus as your data source.

All your favorite parts of Medium are now in one sidebar for easy access.



The screenshot shows the Grafana web interface for importing a dashboard. The browser address bar shows 'dashboard/import'. The page title is 'Importing dashboard from Grafana.com'. It displays metadata for a dashboard published by 'galexrt' and updated on '2018-06-26 14:57:03'. Under the 'Options' section, the 'Name' field is 'Podinfo Observability Dashboard', the 'Folder' is 'Dashboards', and the 'Unique Identifier (UID)' is 'ypFZFgvmz'. The data source is set to 'prometheus-api' with 'prometheus' selected. At the bottom are 'Import' and 'Cancel' buttons.

dashboard/import

Home > Dashboards > Import dashboard

Search or jump to...

Importing dashboard from Grafana.com

Published by galexrt

Updated on 2018-06-26 14:57:03

Options

Name

Podinfo Observability Dashboard

Folder

Dashboards

Unique Identifier (UID)

The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.

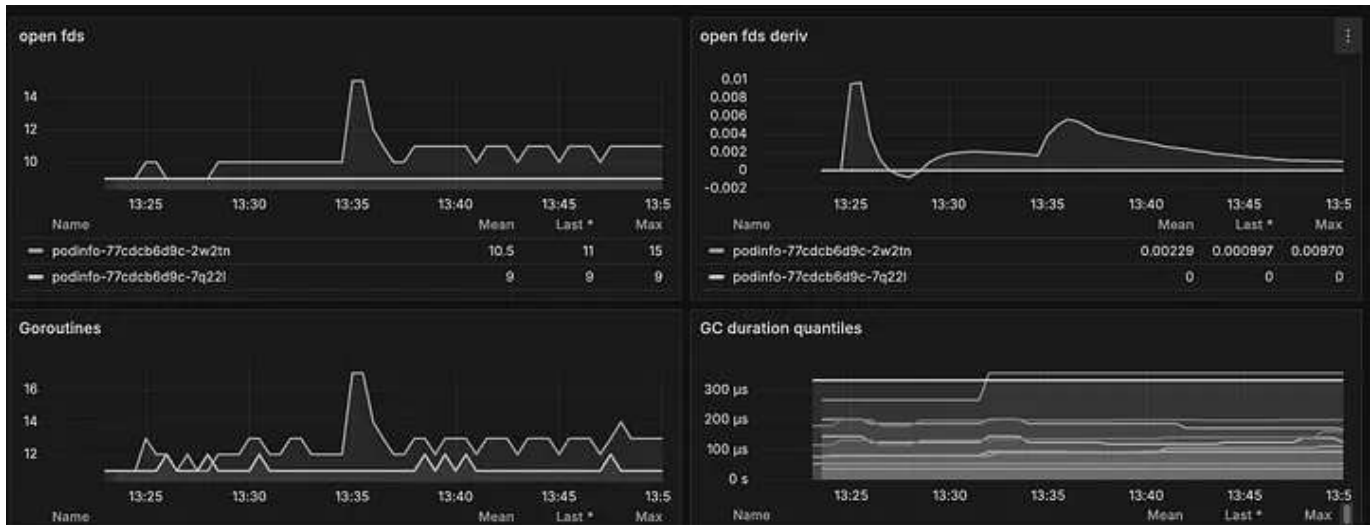
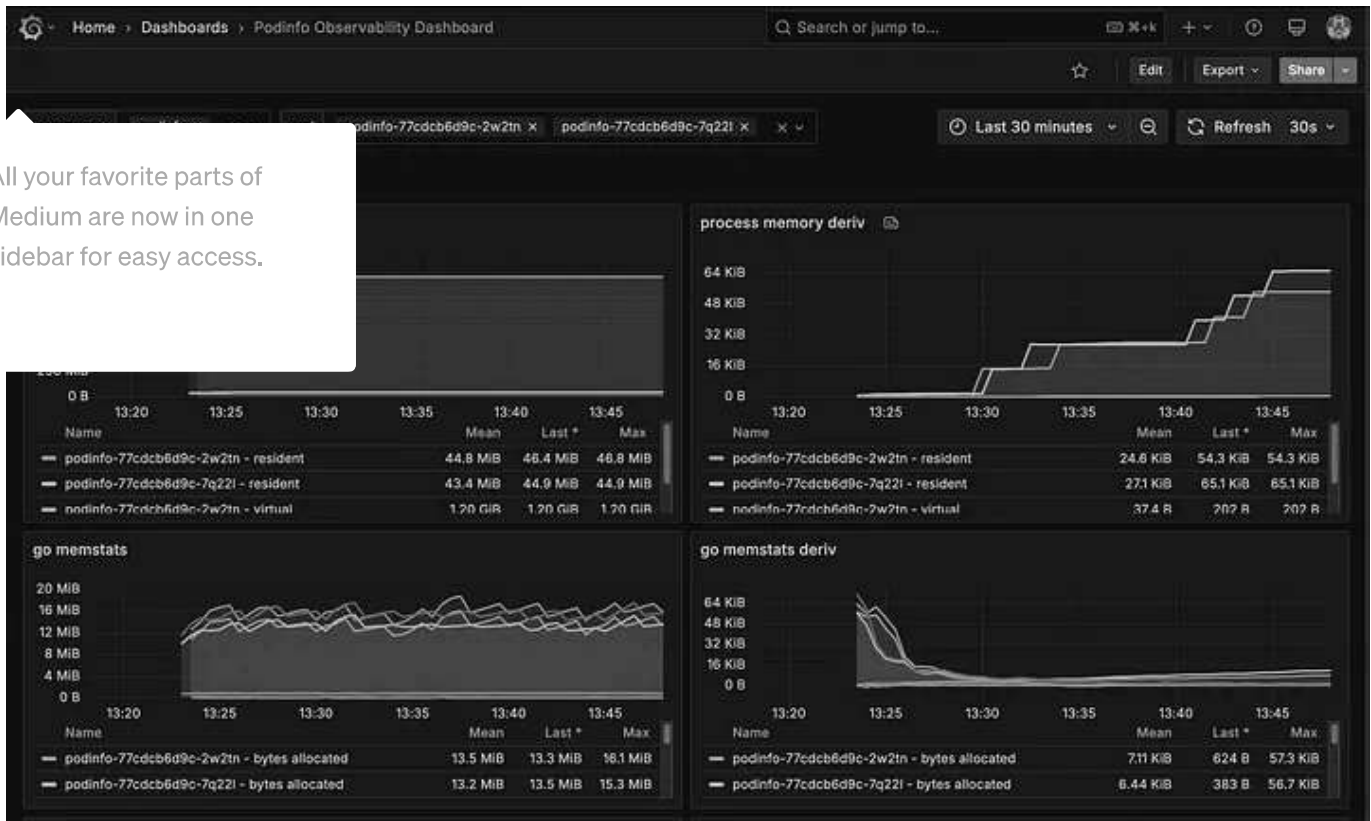
ypFZFgvmz [Change uid](#)

prometheus-api

prometheus

Import Cancel

4. Hit Import.



You'll now see live runtime metrics from your Podinfo app, including memory usage, goroutines, garbage collection stats, and more — everything you'd expect in a real-world Go service!

We're tracking key runtime metrics from each Podinfo instance:

- **Memory usage:** both resident and virtual memory over time

All your favorite parts of
Medium are now in one
sidebar for easy access.

Heap memory usage: monitored via Go's internal memory stats

Probes: useful for identifying resource exhaustion

via Go routines per pod, helping us spot leaks or spikes

- **Garbage collection duration:** GC pauses that can impact performance

8.2: Create a Custom Grafana Dashboard for HTTP Metrics

Let's now build a custom dashboard tailored for HTTP behavior — **the real-world signals** you care about most.

We'll visualise:

-  Total HTTP requests
-  Failed requests (4xx/5xx)
-  Request duration (latency)
-  Success rate percentage

Dashboard Setup

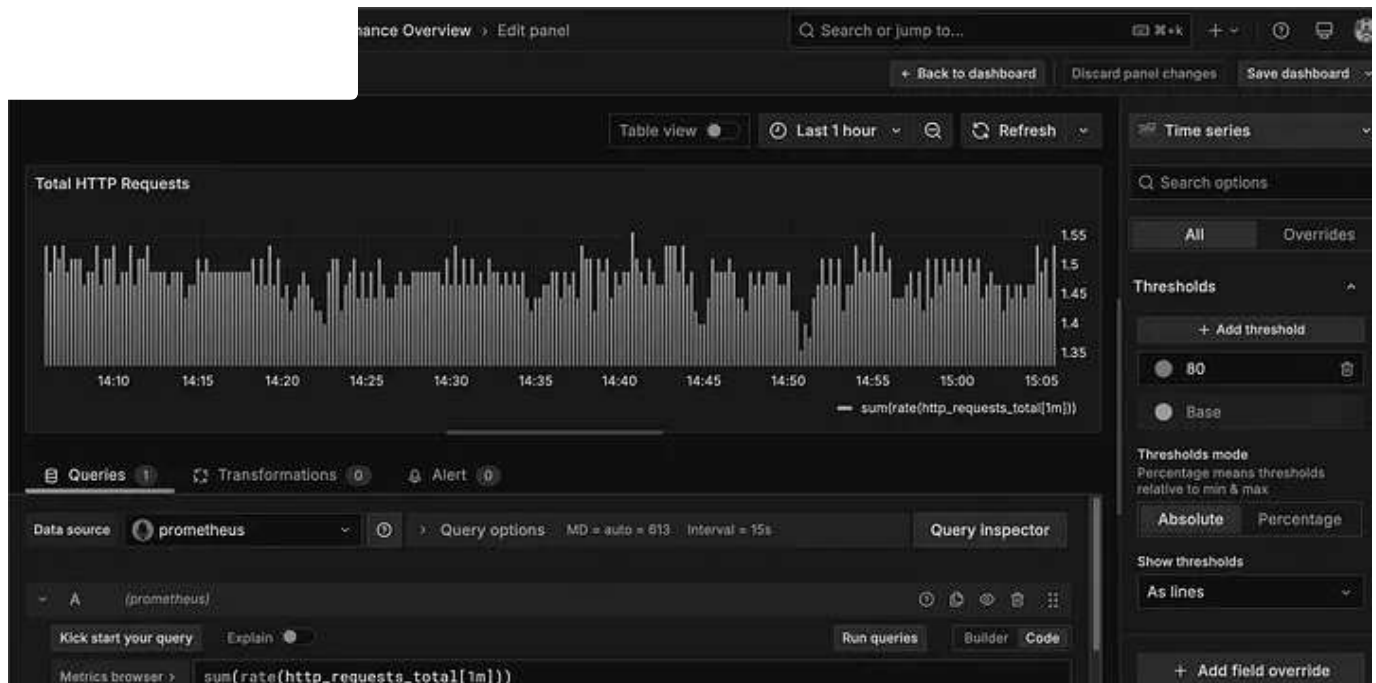
1. Open Grafana at <http://localhost:3000>
2. In the left sidebar, click the “+” → **Dashboard**
3. Click “Add a new Visualisation”

Panel 1: Total HTTP Requests

- Query:

```
sum(rate(http_requests_total[1m]))
```

All your favorite parts of Medium are now in one sidebar for easy access.

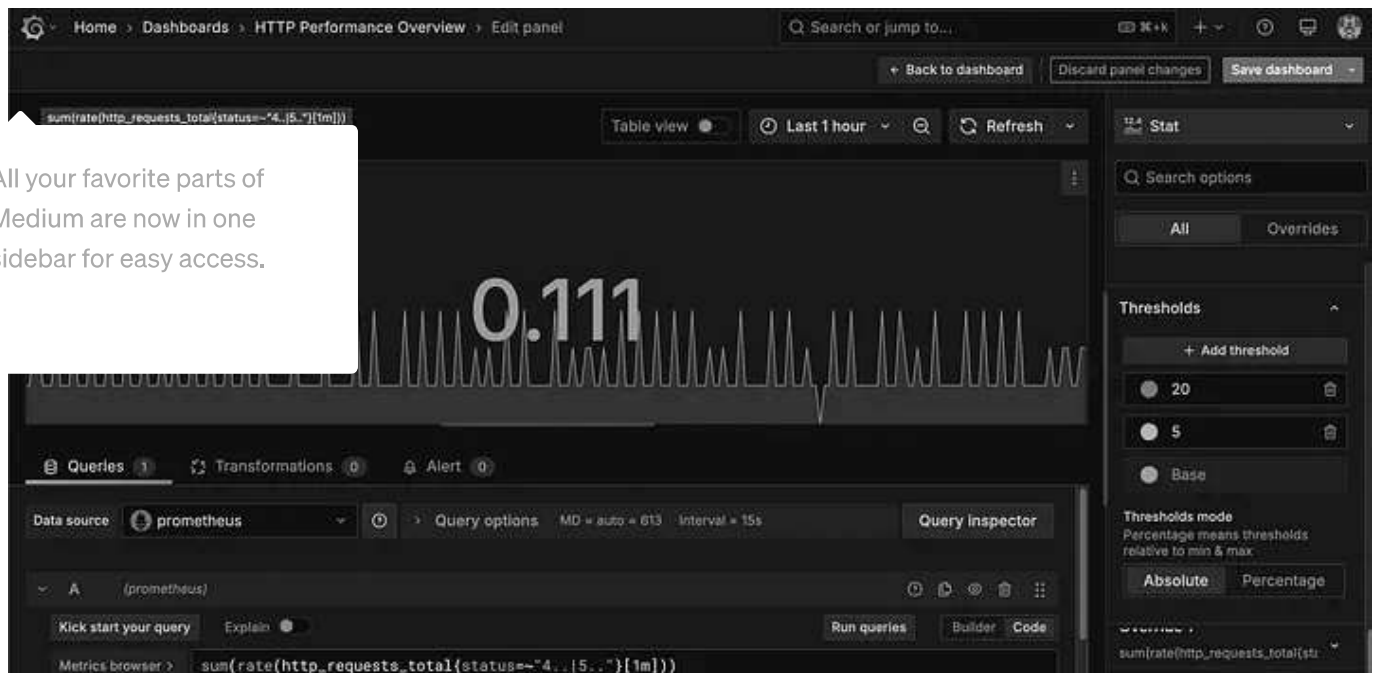


- **Legend:** Total Requests
- **Visualization:** Time series

⚠ **Panel 2: Failed Requests**

- **Query:**

```
sum(rate(http_requests_total{status=~"4..|5.."}[1m]))
```



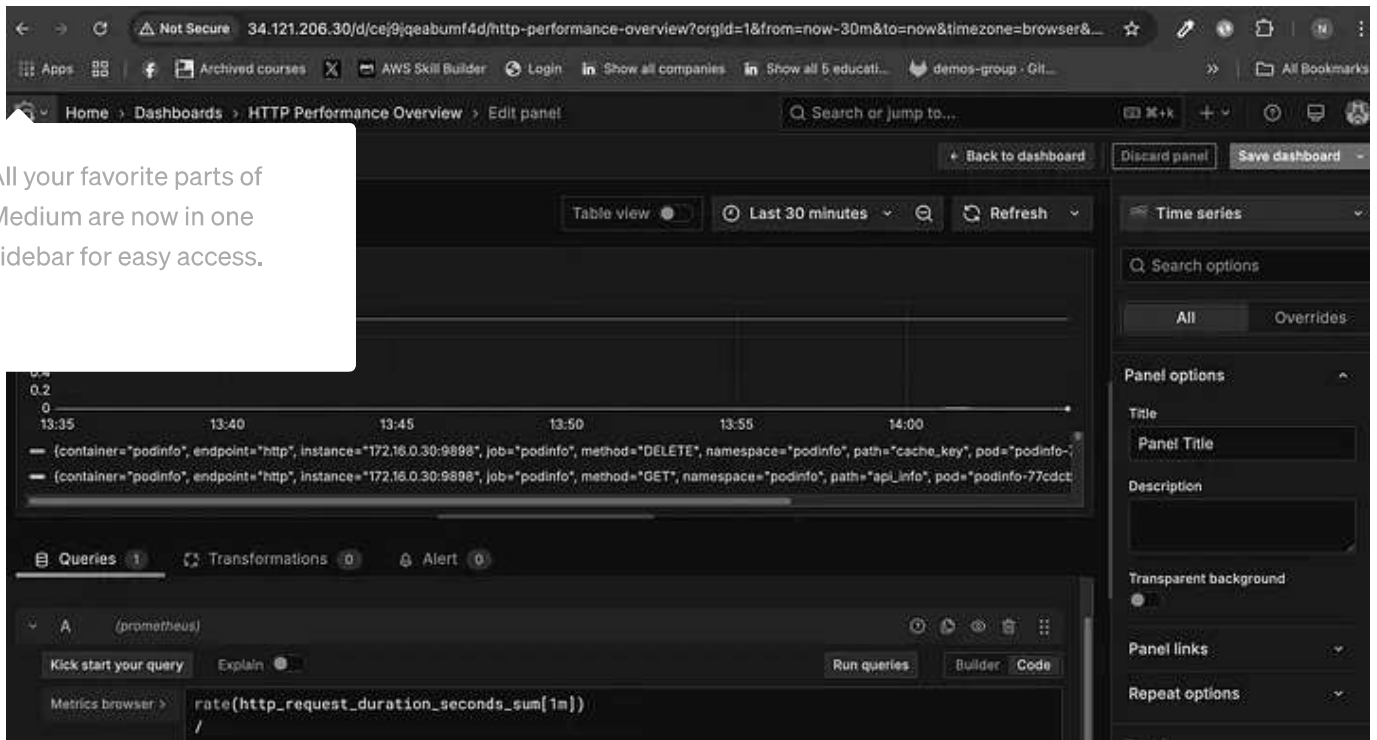
- Legend: Failed Requests
- Use a red color to highlight errors visually

🕒 Panel 3: Average Request Duration

- Query:

```
rate(http_request_duration_seconds_sum[1m])  
/  
rate(http_request_duration_seconds_count[1m])
```

All your favorite parts of Medium are now in one sidebar for easy access.



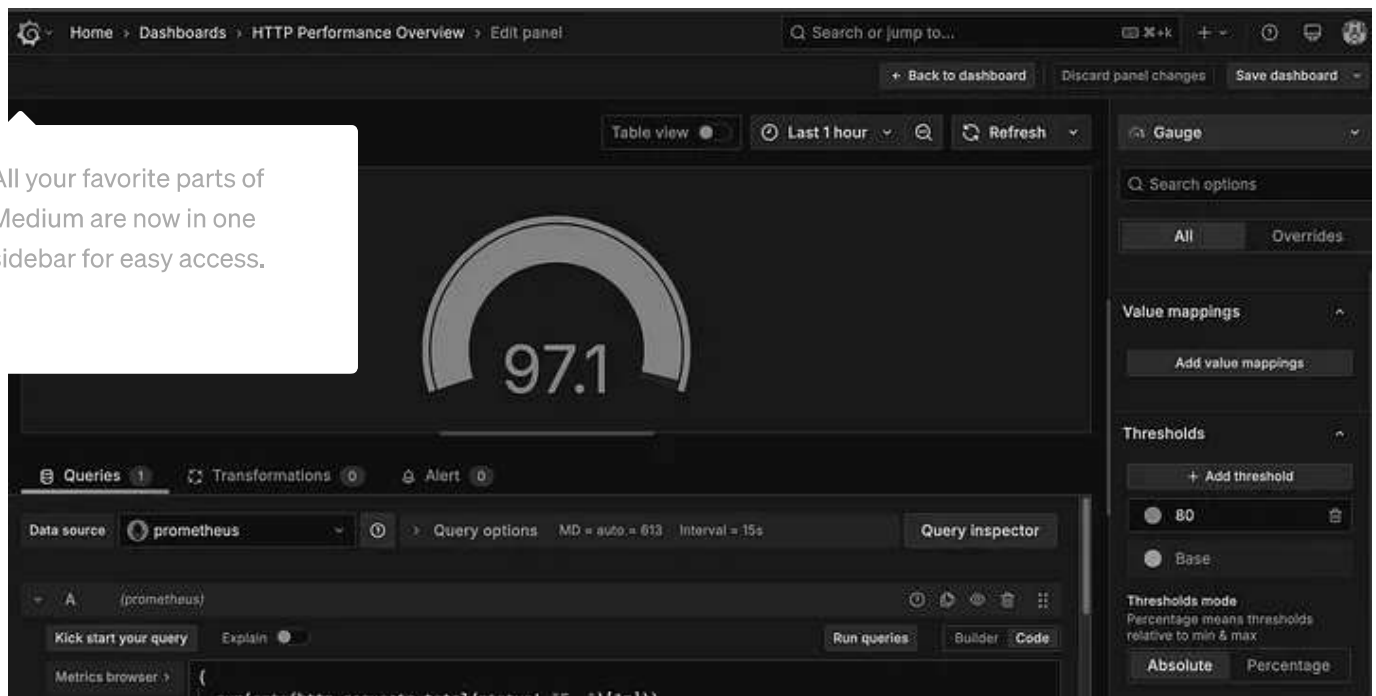
- Legend: Avg Duration (s)
- Set Y-axis unit to seconds

Panel 4: Success Rate (%)

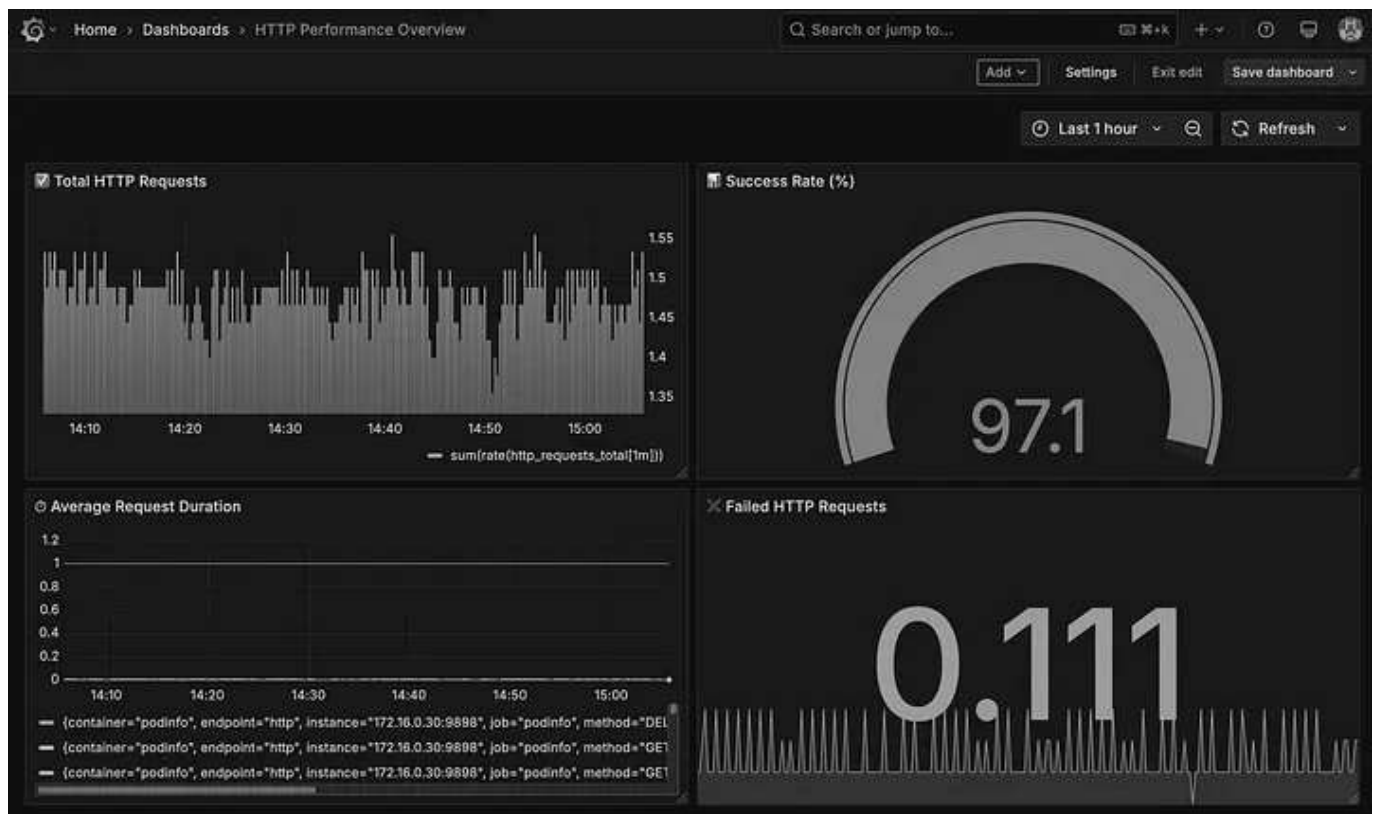
- Query:

```
(  
  sum(rate(http_requests_total{status!~"5.."}[1m]))  
  /  
  sum(rate(http_requests_total[1m]))  
) * 100
```

All your favorite parts of Medium are now in one sidebar for easy access.



- **Legend:** Success Rate (%)
- **Y-axis unit:** Percentage
- Add a **threshold line** at 99% if you want to monitor SLOs



Once completed, you'll have a **live dashboard** showing **production-grade HTTP observability** in real time. This gives you critical insight into traffic, alth — all from Prometheus metrics.

All your favorite parts of
Medium are now in one
sidebar for easy access.

ustom Dashboard for Logs with Loki

Now that metrics are flowing, let's take it one step further and centralize **application logs** using **Grafana Loki**. This helps you investigate real-time issues, correlate logs with metrics, and debug with precision — just like in production setups.

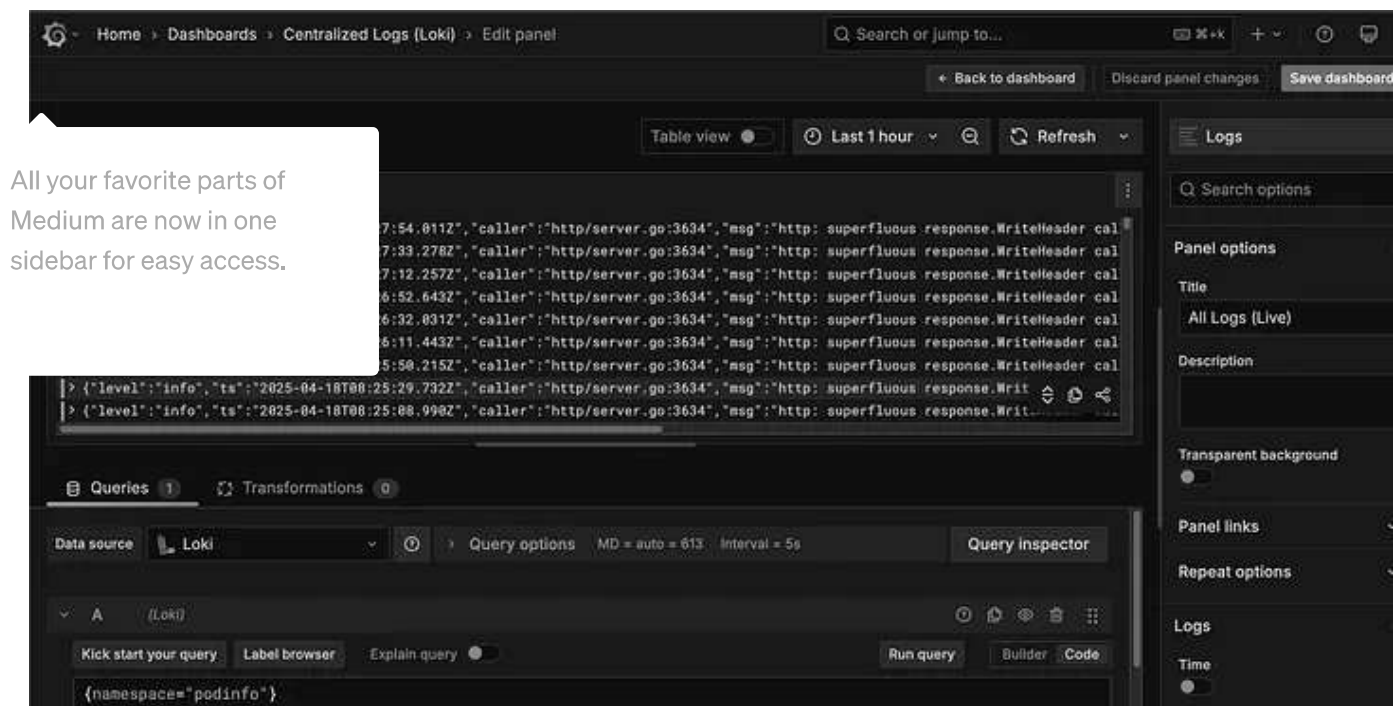
Step-by-Step: Build Your Logging Dashboard

1. Go to Grafana Dashboards
2. Click **+** Create → Dashboard
3. Add a Panel
4. Click “Add a new panel”
5. Select Data Source: Loki

Panel 1: All Logs from Podinfo

- Query:

```
{app_kubernetes_io_name="podinfo"}
```



- **Title:** All Logs (Live)
- Use the Logs visualization.
- Enable **Live Mode** if desired for real-time streaming.

● **Panel 2: Error Logs**

Query:

```
{namespace="podinfo"} |= "error"
```

Title: Error Logs

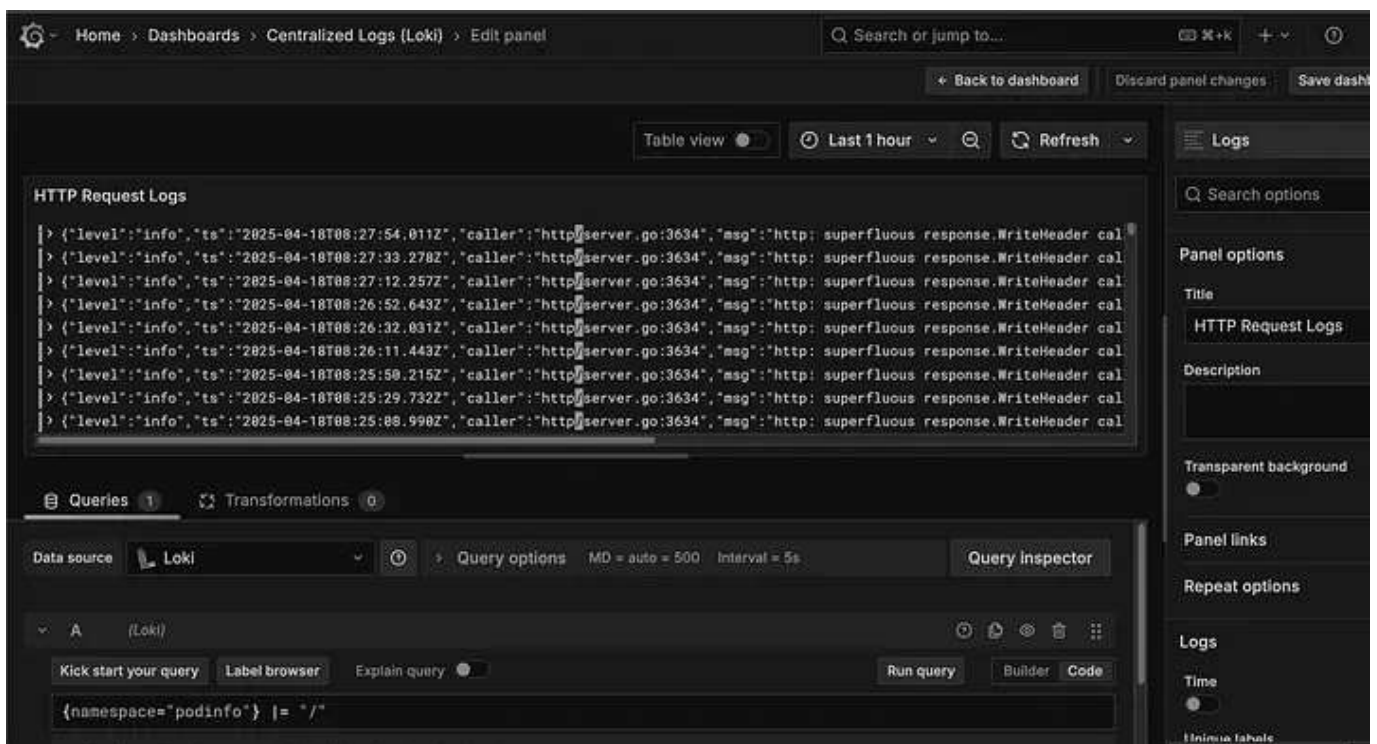
● **Panel 3: Warning Logs**

```
{namespace="podinfo"} |= "warn"
```

All your favorite parts of
Medium are now in one
sidebar for easy access.

Panel 4: HTTP Request Logs

Query:



Home > Dashboards > Centralized Logs (Loki) > Edit panel

Search or jump to...

Back to dashboard Discard panel changes Save dashboard

Table view Last 1 hour Refresh

HTTP Request Logs

```
> {"level":"info","ts":"2025-04-18T08:27:54.011Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
> {"level":"info","ts":"2025-04-18T08:27:33.278Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
> {"level":"info","ts":"2025-04-18T08:27:12.257Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
> {"level":"info","ts":"2025-04-18T08:26:52.643Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
> {"level":"info","ts":"2025-04-18T08:26:32.031Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
> {"level":"info","ts":"2025-04-18T08:26:11.443Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
> {"level":"info","ts":"2025-04-18T08:25:50.215Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
> {"level":"info","ts":"2025-04-18T08:25:29.732Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
> {"level":"info","ts":"2025-04-18T08:25:08.998Z","caller":"httpserver.go:3634","msg":"http: superfluous response.WriteHeader call"}
```

Queries 1 Transformations 0

Data source: Loki Query options: MD = auto = 500 Interval = 5s Query Inspector

A (Loki)

Kick start your query Label browser Explain query Run query Builder Code

```
{namespace="podinfo"} |= "/"
```

Logs

Search options

Panel options

Title: HTTP Request Logs

Description

Transparent background

Panel links

Repeat options

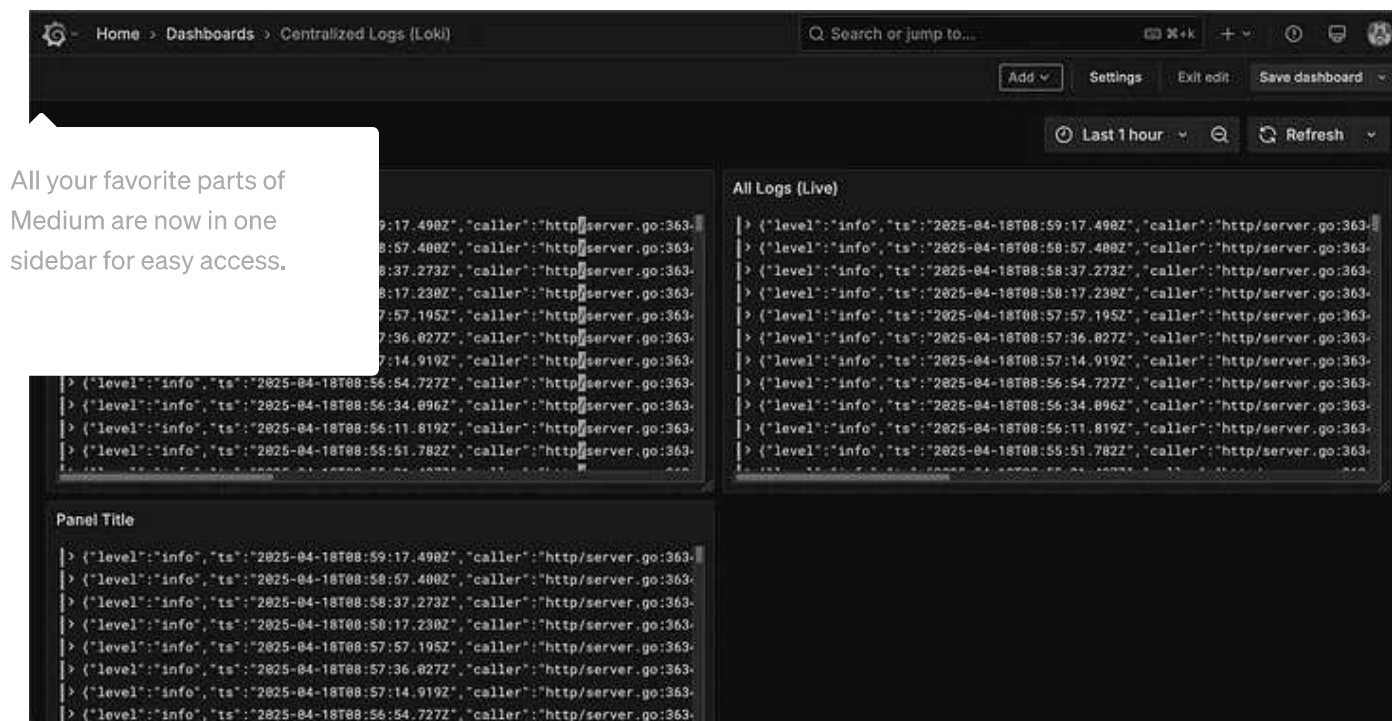
Logs

Time

Minimum labels

Title: HTTP Requests

 **Note on Log Types:** Since the podinfo application is a lightweight demo app, it doesn't emit detailed business-level logs like: login/login failed, access denied, user activity, database queries



However, in a real-world app, you can extend your log pipeline by:

- Structuring logs with log levels and context fields
- Adding app-specific messages for each major operation
- Parsing them in Promtail or Fluent Bit with json stages to extract log level and labels

8.3 : Monitor Kubernetes Cluster with Prebuilt Grafana Dashboards

When you deploy kube-prometheus-stack, it automatically ships with a set of prebuilt dashboards — designed by the community and trusted by production teams. These dashboards give you instant observability across your Kubernetes cluster, nodes, and critical components.

How to Access These Dashboards

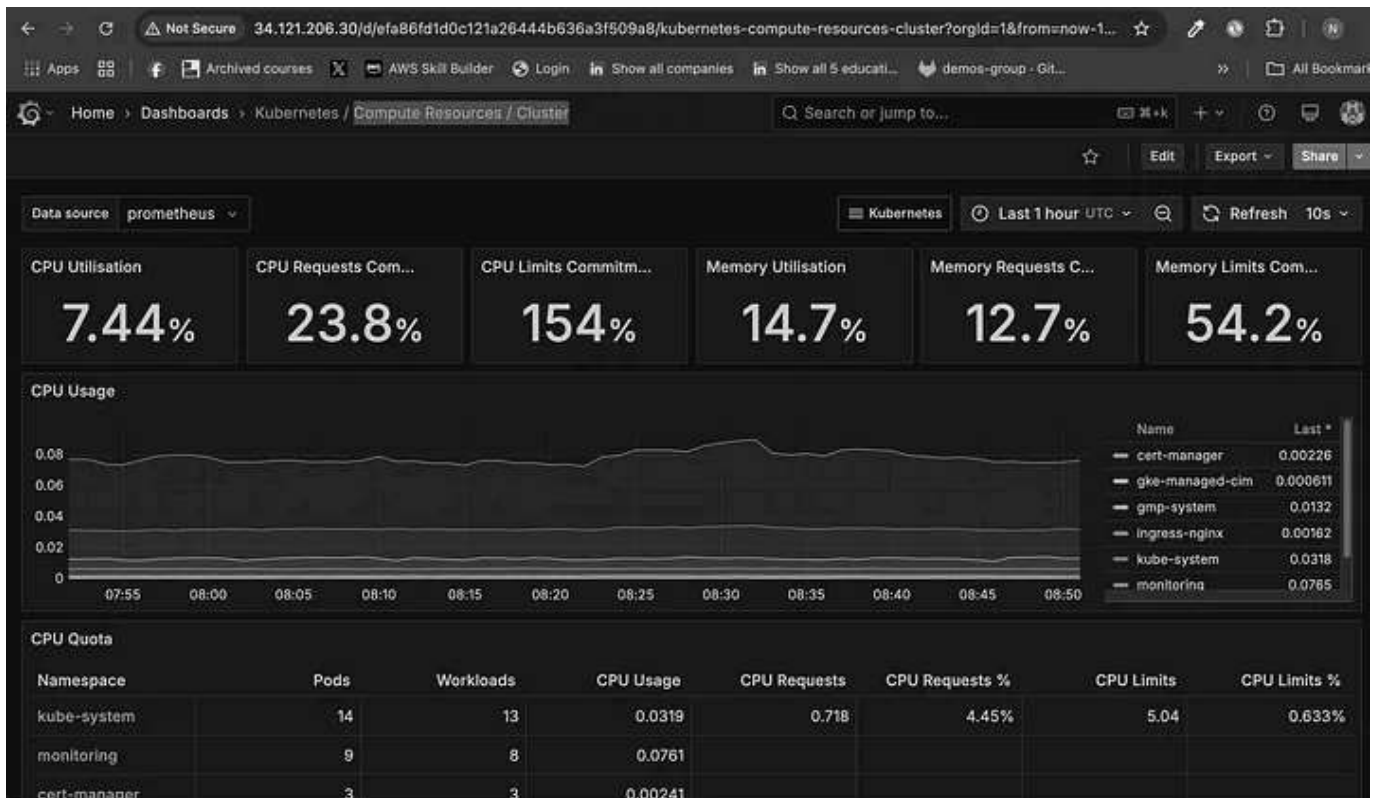
1. Open Grafana at <http://34.121.206.30/dashboards>

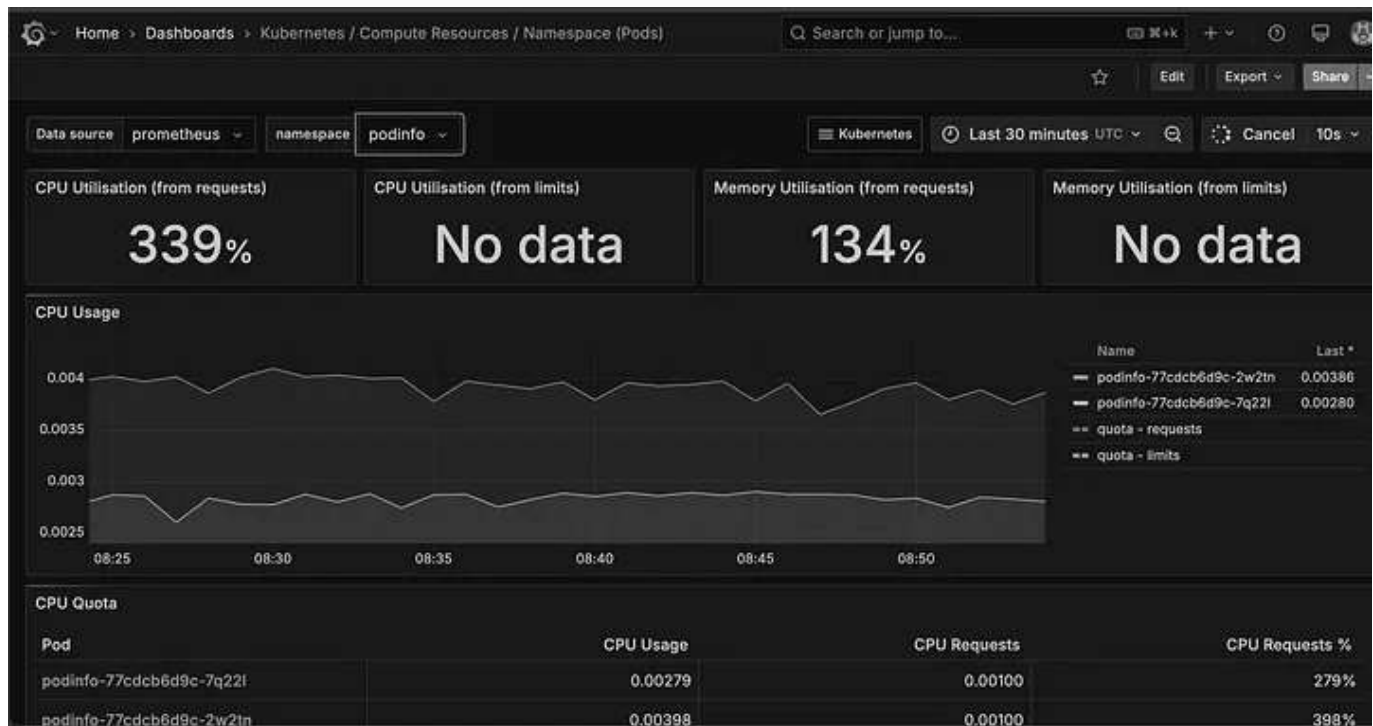
2. Go to the left sidebar → “Dashboards” → “Browse”

Now you can find a list of dashboards categorized by folders (e.g. Compute Resources, kubernetes-mixin, node-exporter-mixin, etc.)

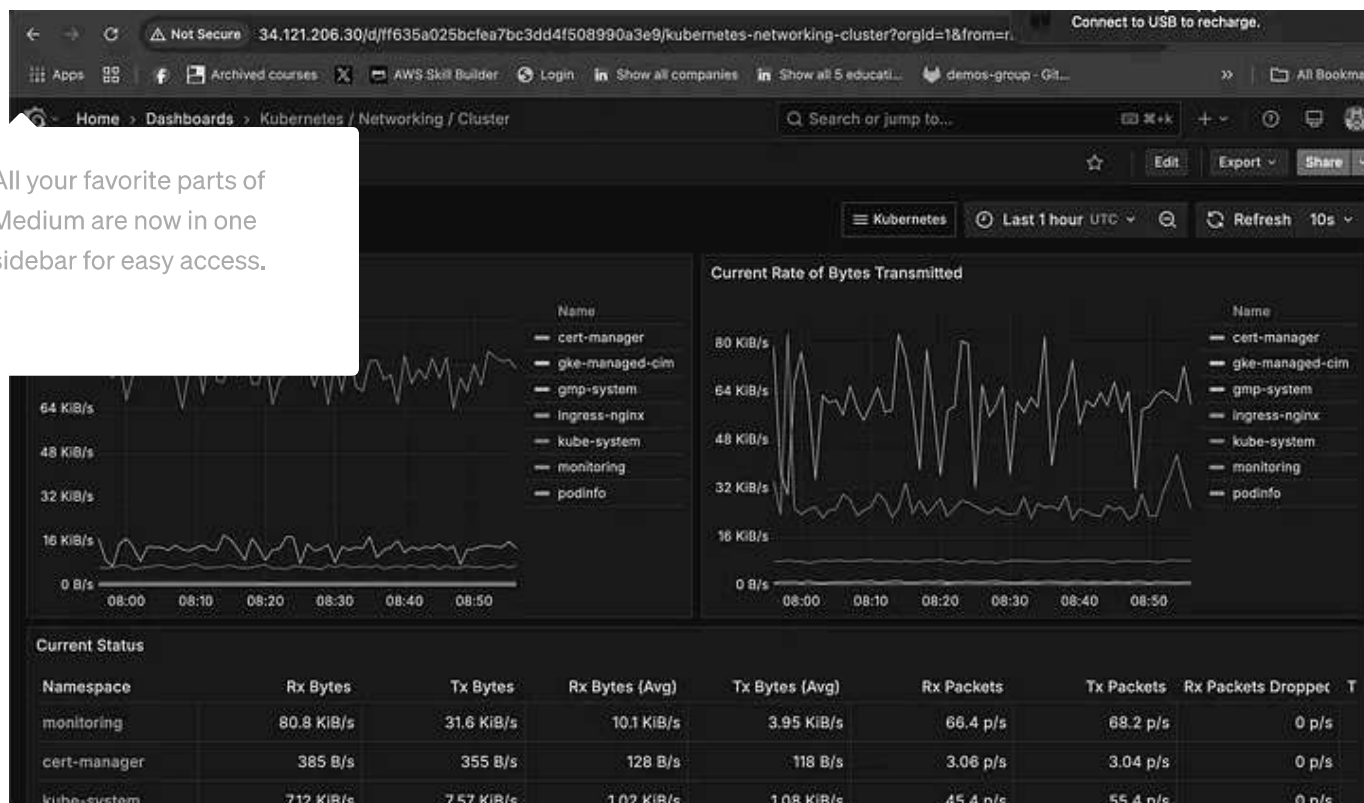
All your favorite parts of Medium are now in one sidebar for easy access.

board to explore metrics, set filters, or customize panels





All your favorite parts of Medium are now in one sidebar for easy access.



💡 These dashboards are automatically provisioned under the **Dashboards** folder when you deploy kube-prometheus-stack.

🎯 What You Can Monitor at Cluster Level

- Node pressure (CPU throttling, memory saturation)
- Disk IO & PVC health
- Network traffic by pod/workload
- Scheduler latency & pending pod queues
- Control plane metrics (API server, scheduler, kube-proxy)
- System-level metrics (load, CPU idle, context switches)

[Open in app](#) ↗

on Kubernetes — the kind used by real DevOps and SRE teams around the

All your favorite parts of
Medium are now in one
sidebar for easy access.

Now Helm charts, Prometheus, Loki, and Grafana, you now

 **Dashboards** to monitor app health, performance, and traffic

-  **Structured Logs** for debugging, auditing, and tracing behavior
-  **Prebuilt Kubernetes Dashboards** for deep cluster introspection
-  **Real-World Traffic Simulation** to validate your monitoring setup in action

This setup gives you everything you need to:

- Detect performance regressions 
- Spot failures early 
- Investigate issues with precision 
- Correlate logs and metrics effortlessly  + 
- And lay the groundwork for SLIs, SLOs, and alerts

What's Next?

In the **next phase of this journey**, I'll be implementing the following to take observability to the next level:

✓ Alertmanager + Custom Alert Rules

All your favorite parts of
Medium are now in one
sidebar for easy access.

able alerts that detect anomalies before users do.

Integration

To unlock full **distributed tracing** across services, ideal for debugging complex interactions

✓ GitOps with ArgoCD

To manage infrastructure and application delivery using **declarative, version-controlled workflows**.

✓ Real Microservices Expansion

I'll replace the demo app with production-style **Node.js microservices**, tracking full request lifecycles and user journeys across services.

These upgrades will be featured in a **new project**, offering a complete, production-like environment for DevOps, SRE, and platform engineering use cases.

Stay tuned — this is just the beginning.

DevOps

Monitoring

Observability

Grafana

Prometheus

All your favorite parts of Medium are now in one sidebar for easy access.

leamul Kabir Emon

following

Follow

Helping businesses Deploy 2x Faster, Cut 40% Cloud Costs & Stay Secure | DevOps Engineer • AWS • Azure • GCP • Penetration Testing | Cloud & Security Expert

No responses yet

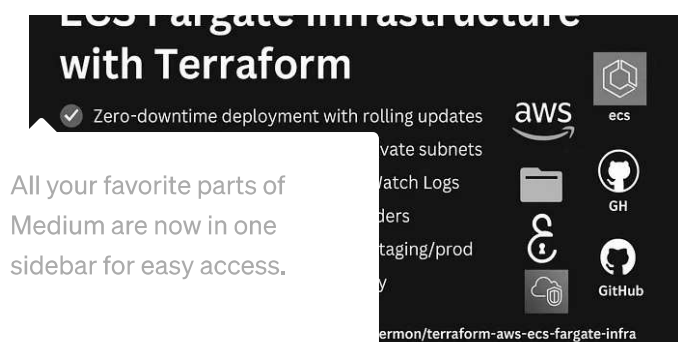


chanchal sakarde

What are your thoughts?



More from Neamul Kabir Emon



 Neamul Kabir Emon

Build a Production-Grade AWS ECS Fargate Cluster with Terraform...

Project Goal

Apr 22  15  1



 Neamul Kabir Emon

Building a Production-Ready GKE Cluster with Terraform, Helm &...

 Overview

Apr 5

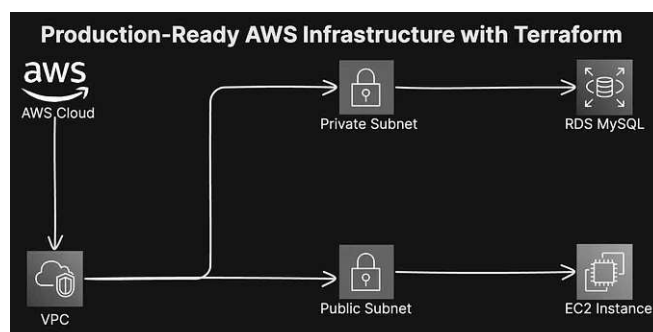


 Neamul Kabir Emon

Build a Real-World DevOps Pipeline with GitHub Actions, Terraform,...

Introduction

Apr 16  50  1



 Neamul Kabir Emon

From Zero to Production: Build Scalable AWS Infrastructure Usin...

Introduction

Apr 11  1



See all from Neamul Kabir Emon

All your favorite parts of Medium are now in one sidebar for easy access.

from Medium



 Rafael Durelli

End-to-End Observability with Kubernetes, Prometheus & Grafa...

TL;DR—In this hands-on guide you will 1. Spin up a local Kubernetes cluster with Kind...

Jun 6  7



 Bhavyansh

Prometheus + Grafana: The Monitoring Setup That Caught 80...

How a simple Prometheus and Grafana setup transformed our incident response from...

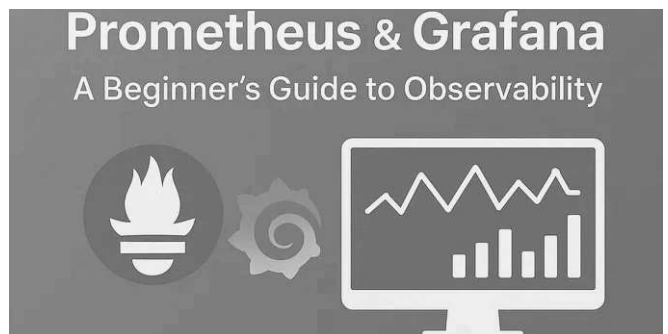
★ Sep 28  21



 Venkata Ashok Kumar Boyina

Full Stack Observability with Grafana, Prometheus, Loki, Temp...

In modern DevOps and cloud-native environments, observability is an essential...



 Omolade Akinwumi

Prometheus Et Grafana

Prometheus & Grafana: A Beginner's intro to Observability

May 28 🖱️ 1 💬 1



Jul 18



All your favorite parts of Medium are now in one sidebar for easy access.



AWS In AWS in Plain English by Harshal Jethwa

Kubernetes Logging and Monitoring with Grafana, Loki,...

In this guide, we'll set up centralized logging and monitoring in a Kubernetes cluster usin...

4d ago



Jirapong P

Complete Beginner's guide to Grafana Operator

Learn How to use Grafana Operator in kubernetes

Oct 7



See more recommendations